SHL Assessment Recommender

Implementation report for the conversational SHL Individual Test Solutions assistant.

Scope	Stateless FastAPI service with grounded recommendations, refinement, comparison, and refusal behavior.
Catalog	Normalizes the SHL JSON catalog endpoint and caches a processed dataset plus FAISS index.
UI	A responsive single-page interface is served at the root route and calls /chat directly.
Deployment	Dockerized with optional Gemini, Anthropic, or OpenRouter provider wiring through environment variables.

Architecture

Stateless conversation history is re-sent on every request.

The backend keeps conversation state out of the server. Each request includes the full message transcript, which the planner uses to decide whether to clarify, recommend, refine, compare, or refuse.

Key decisions

- Hybrid retrieval uses local TF-IDF plus FAISS, with metadata boosts for role, seniority, duration, and test-family signals.
- The catalog source of truth is the SHL JSON endpoint. Records are cached as raw and normalized JSON to keep startup fast.
- Prompt templates live in app/prompts/ so the behavior can be tuned without hardcoding prompt text into the service.
- Recommendation output stays deterministic and schema-compliant even when an LLM is configured for reply polishing.

Verification

Unit + integration tests	13 passing tests covering schema, recommendation, refinement, comparison, refusal, and retrieval smoke checks
UI	Responsive root page with quick prompts, local conversation history, and recommendations rendered as cards.
PDF report	Generated with reportlab and verified via pdftoppm rendering.

Deployment and operational notes

The project ships with a Dockerfile, Compose file, Render and Railway configs, and a README that explains the Gemini, Anthropic, and OpenRouter environment variables. The same container can serve both the UI and the API on port 7860.

Gemini is optional but supported. If GEMINI_API_KEY is present and the provider mode is auto, the app prefers Gemini for final reply polishing. If no key is configured, the service falls back to a deterministic template path.

Limitations

The local embedding backend is intentionally lightweight and lexical-heavy. That makes the app fast and portable, but a dedicated semantic embedding model could improve recall on harder, more abstract traces.

Future improvements

- Swap in a hosted or local semantic embedding model once the runtime dependency set is stable.
- Add trace-driven prompt tuning and per-intent evaluation reports.
- Expand alias handling for acronyms and product-family shorthand in the catalog.

This report was generated directly from the implementation state in the workspace and is intended to be concise enough for the assignment submission limit.